

操作计划追踪系统设计

操作计划追踪系统 — 设计文档

1. 背景与目标

工作台(Workbench)每日生成“明日计划”(buy/sell/watch), 但目前没有系统追踪这些计划的实际执行效果。目标是自动追踪每个计划第二天(下一个交易日)的涨跌表现, 统计成功率, 辅助优化3L买卖点体系。

核心问题

- 我们制定的买入/卖出计划, 实际第二天涨了还是跌了?
- 哪些类型的条件成功率更高? (上涨趋势上行 vs 缩量整理 vs 区底企稳)
- 整体成功率和平均盈亏比是多少?

2. 数据源

现有数据

- 工作台计划: data/private/workbench/{date}.json
 - plan.buy[] / plan.sell[] / plan.watch[]
 - 每个 item: {stock: "杭齿前进(601177)", condition: "上涨趋势·上行", stop_loss: 17.6, ...}
- 股票代码映射: data/all_a_stocks.json → {code: name} 双向映射
- 个股K线: 通过 on_demand_stock.py 按需拉取 (或从已有的 all_stocks 缓存中读取)

关键观察

- 计划中的 stock 字段已包含股票代码, 格式为 股票名(股票代码)
- 可通过 resolve_stock() 或正则提取代码
- 个股日K线数据可通过腾讯接口按需获取

3. 设计方案

3.1 数据模型

```
{
  "plans": [
    {
      "plan_date": "2026-05-28",
      "type": "buy",
      "stock": "杭齿前进",
      "code": "601177",
      "condition": "上涨趋势·上行",
      "condition_category": "上涨趋势",    // 条件大类（用于统计分组）
    }
  ]
}
```

```

        "condition_detail": "上行", // 条件细类
        "stop_loss": 17.6,
        "stop_loss_pct": 5.22,

        "plan_close": 18.56, // 计划当天的收盘价
        "next_date": "2026-05-29",
        "next_open": 18.70,
        "next_close": 18.90,
        "next_high": 19.20,
        "next_low": 18.50,

        "change_pct": 1.83, // (next_close - plan_close) /
plan_close * 100
        "max_gain": 3.45, // (next_high - plan_close) /
plan_close * 100
        "max_loss": -0.32, // (next_low - plan_close) /
plan_close * 100
        "hit_stop_loss": false, // 盘中是否跌破止损价

        "result": "success", // success | failure | flat(±0.5%内)
| no_data
        "executed": null, // true | false | null(未知, 需用户标记)
        "user_note": "" // 用户备注 (为什么成功/失败)
    }
},
"summary": {
    "total_plans": 7,
    "by_type": { "buy": 5, "sell": 1, "watch": 1 },
    "success": 4,
    "failure": 3,
    "flat": 0,
    "success_rate": 57.1,
    "avg_gain_pct": 2.1, // 成功计划的平均涨幅
    "avg_loss_pct": -1.5, // 失败计划的平均跌幅
    "best_gain": 5.2,
    "worst_loss": -3.1,
    "win_loss_ratio": 1.4 // avg_gain / |avg_loss|
},
"by_condition": {
    "上涨趋势": { "total": 4, "success": 3, "rate": 75.0 },
    "区间震荡": { "total": 2, "success": 1, "rate": 50.0 },
    ...
},
"last_updated": "2026-05-30T10:00:00"
}

```

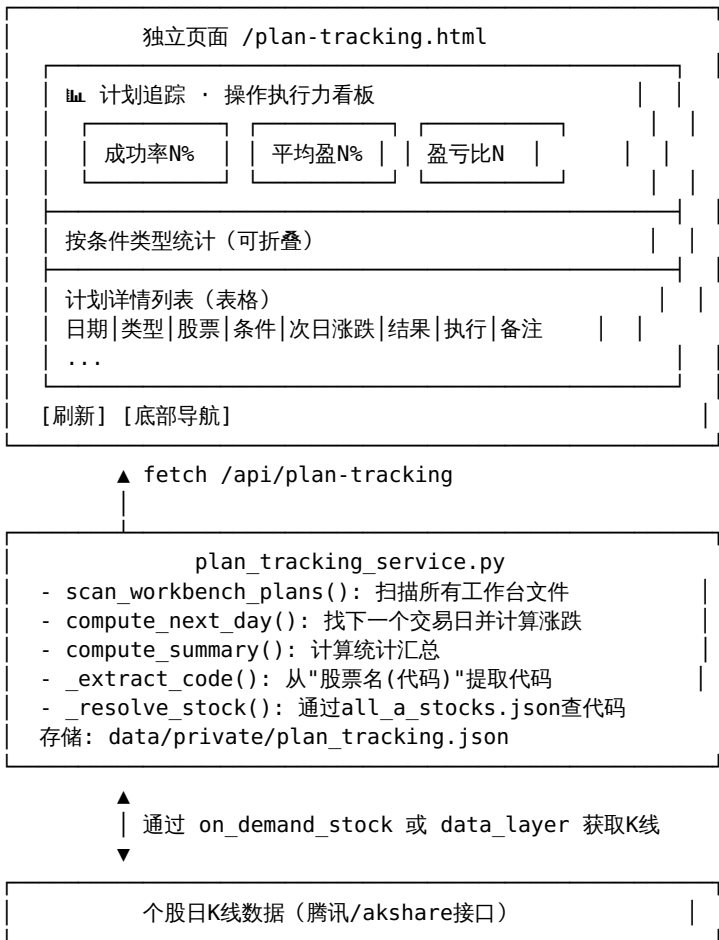
3.2 成功/失败判定规则

买入计划: | 条件 | 判定 | 说明 | |——|——|——| | 次日收盘 > 计划日收盘 +0.5% | ✓ success | 上涨超过摩擦成本 | | 次日收盘 < 计划日收盘-0.5% | ✗ failure | 下跌超过摩擦成本 | | 介于±0.5%之间 | — flat | 基本平盘 | | 无次日数据 (最后一交易日) | ? no_data | 待后续补充 |

卖出计划 (反向): | 条件 | 判定 | |——|——|——| | 次日收盘 < 计划日收盘-0.5% | ✓ success (卖对了) | | 次日收盘 > 计划日收盘+0.5% | ✗ failure (卖飞了) |

观察计划: 不参与成功率统计, 仅展示次日涨跌供参考

3.3 系统架构



3.4 API 设计

```
GET /api/plan-tracking
返回: {
  plans: PlanItem[],           // 所有计划 + 追踪结果
  summary: Summary,           // 统计汇总
  by_condition: {},           // 按条件类型分组统计
  last_updated: "2026-05-30T10:00:00"
}

POST /api/plan-tracking/annotate
参数: {
  plan_id: "2026-05-28-buy-0",
  executed: true/false,
  user_note: "盘中触发买点, 按计划执行"
}
返回: { success: true }

POST /api/plan-tracking/refresh
触发重新计算所有未完成的计划追踪
返回: { updated: 5, no_data: 1 }
```

3.5 前端页面

新增独立页面 PlanTracking.tsx，通过底部导航访问，路径 /plan-tracking.html。

页面结构：

- 1. 顶部统计卡片行（一行3个关键指标）：| 成功率 | 平均盈利 | 盈亏比 | |——|——|——|——| | 大号百分比数字 | ✓ +2.1% | 1.4 倍 | | 成功N/总N | 最佳 +5.2% | 最差-3.1% |
- 2. 按条件类型分组统计（可折叠）：
 - 按条件类型
 - 上涨趋势·上行：3/4=75% +2.8%
 - 上涨趋势·缩量整理：0/1=0% -1.2%
 - 区间震荡·区间底部：1/2=50% +0.5%
- 3. 计划详情列表（表格）：

日期	类型	股票	条件	次日涨跌	结果	执行
备注						
05-28	buy	杭齿前进	上涨趋势·上行	+1.83%	✓ 成功	待标记
05-28	buy	中天科技	区间震荡·区底	-2.10%	✗ 失败	待标记
05-26	buy	广钢气体	中继买点	+0.55%	- 平盘	待标记
- 4. 底部操作栏：刷新数据、底部导航
- 5. 无计划时的状态：纯文字提示“暂无计划数据，请先在工作台制定明日计划”

3.6 执行时机

- 按需计算（默认）：访问页面时触发计算，结果写入缓存文件
- 增量更新：只计算之前没有追踪结果的计划
- 不依赖cron：前端触发+后端计算，避免cron中增加的额外负担

4. 不做（v1范围外）

项目	原因
第3天/第5天追踪	用户明确先看第2天
自动标记“已执行”	需要接入交易系统，超出v1范围
实盘成交价追踪	没有券商API接入
watch/观察计划统计	观察计划不确定操作方向
板块/大盘同期对比	v2再扩展
图表可视化（K线标注）	文本表格足够，v2可加

5. 数据流示例

用户在工作台制定计划（2026-05-28）

- plan.buy[]: [杭齿前进(601177), 光迅科技(002281), ...]
- 保存到 data/private/workbench/2026-05-28.json

次日/以后查看追踪页面（2026-05-30）

- GET /api/plan-tracking

→ plan_tracking_service:
1. 扫描所有工作台文件
2. 对每个计划的股票, 找下一交易日的K线
3. 计算涨跌幅, 判定成功/失败
4. 汇总统计
→ 返回结果 → 前端渲染

6. 文件清单

新增:

server/backend/services/plan_tracking_service.py	- 追踪计算服务
server/backend/api/plan_tracking.py	- API路由
server/frontend/src/pages/PlanTracking.tsx	- 独立追踪页面
server/frontend/src/pages/PlanTracking.css	- 样式
docs/plan-tracking-design.md	- 本设计文档

修改:

server/server.py	- 注册路由 + HTML
------------------	---------------

页路由

server/frontend/src/lib/NavBar.tsx	- 底部导航加"计划追
------------------------------------	-------------

踪"入口

数据:

data/private/plan_tracking.json	- 追踪结果缓存 (自
---------------------------------	-------------

动生成)

7. 分支策略

- 分支名: feature/plan-tracking
- 基于 master 创建
- 先这个设计文档 + PDF 审阅
- 然后 TDD 开发
- 完成后合并回 master